

Fiche d'Activité : Refactoring et Optimisation

0.1. Travaux réalisés

L'objectif de l'intervention était de migrer la logique de validation agronomique du contrôleur vers un service dédié tout en optimisant les performances de traitement.

0.1.1. 1. Architecture Logicielle : Création du `NdoseService`

J'ai procédé au refactoring de la classe `NdoseController` pour extraire la logique métier vers `NdoseService`.

- Simplification du Controller : Les endpoints (`/eligibilities/validate`, `/engines/getComputationEngines`, etc.) délèguent désormais les appels au service via l'injection de dépendances.
- Responsabilité : Le service centralise les accès aux données (`EntityManager`, `Repositories`) et le traitement algorithmique.

0.1.2. 2. Optimisation de la validation d'éligibilité

Le but de la fonction `getUnknownEligibilities` est de filtrer les demandes utilisateurs pour ne retourner que les combinaisons (Culture, Département, Méthode, Sol) non référencées en base.

- Technologie utilisée : Utilisation de `QueryDSL` pour construire une requête complexe dynamique.
- Performances SQL : Implémentation de jointures explicites (`innerJoin`) entre les entités `Eligibility`, `Crop`, `Soil` et `SoilGroup` pour récupérer l'ensemble des données en un seul appel.
- Traitement Java :
 - Stockage des résultats existants dans un `Set<String>` sous forme de clés concaténées (`code|dept|crop|soil`).
 - Comparaison avec la liste demandée en temps constant ($O(1)$), optimisant ainsi le filtrage des éligibilités inconnues.

0.1.3. 3. Fiabilisation du modèle de données

0.2. État des tests et blocages

- Tests unitaires/intégration : Le code de test (`NdoseControllerTest`) n'a pas encore été modifié. Il échoue actuellement avec une erreur 403 Forbidden.
- Diagnostic (hypothèse) : L'échec est dû à l'activation de la sécurité sur l'endpoint (nécessité d'un utilisateur mocké et gestion du jeton CSRF pour la méthode POST).